

Smart Interviews Contest Prep Sheet ■

■ Overview

Smart Interviews contests test your logical reasoning and algorithmic depth through classic Data Structures and Algorithms (DSA) topics. This sheet highlights the most common question patterns, Java code hints, and quick revision notes to help you perform confidently.

■ Core Topics & Must-Solve Patterns

■ Arrays & Subarrays

- Maximum Subarray Sum (Kadane's Algorithm)
- Subarray with Given Sum (Prefix Sum / HashMap)
- Next Greater Element
- Leaders in an Array
- Trapping Rainwater

■ Strings

- Longest Palindromic Substring
- Anagram Check / Group Anagrams
- Longest Substring Without Repeating Characters (Sliding Window)
- Pattern Search (KMP Algorithm)

■ Recursion & Backtracking

- Generate All Subsets / Power Set (Lexicographical Order)
- Permutations of String / Array
- N-Queens / Sudoku Solver
- Generate Parentheses

■ Linked Lists

- Reverse a Linked List (Iterative & Recursive)
- Detect Loop (Floyd's Cycle Detection)
- Merge Two Sorted Lists
- Find Middle Element

■ Binary Trees / BST

- Inorder / Preorder Traversal
- Lowest Common Ancestor (LCA)
- Validate BST
- Kth Smallest Element in BST

■ Dynamic Programming

- 0/1 Knapsack
- Longest Increasing Subsequence (LIS)
- Coin Change Problem
- Matrix Minimum Path Sum

■ Java Input-Output and Code Hints

Fast Input:

Use `BufferedReader` for fast input and `StringTokenizer` for splitting tokens.

Template:

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
int t = Integer.parseInt(br.readLine());
StringTokenizer st = new StringTokenizer(br.readLine());
int n = Integer.parseInt(st.nextToken());
```

Output Optimization: Use `StringBuilder` for multiple prints, and print once at the end.

Example:

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < n; i++) sb.append(arr[i]).append(' ');
System.out.println(sb.toString());
```

■ 5 Warm-Up Problems Before Contest

- Find Missing Number in 1..N Array
- Count Pairs with Sum K
- Rotate Matrix by 90°
- Check Balanced Parentheses using Stack
- Find Maximum Product Subarray

■ Quick Notes

- Prefer $O(n)$ or $O(n \log n)$ solutions.
- Dry-run corner cases (empty arrays, negatives, duplicates).
- Write helper functions for clarity.
- Revise recursion and dynamic programming patterns before the contest.
- Avoid unnecessary `System.out.print` calls — it slows execution.

Best of luck, Shannu! ■ You've got this — stay calm, stay logical, and code smart! ■